

Project Progress Report on

Blockchain-Based Freelancing Platform Using MERN

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by:

Student Name 1 : Kunal Malik

University Roll No. : 2021812

Student Name 2 : Arun Jethi

University Roll No. : 2021131

Student Name 3 : Vishal Sharm

University Roll No. : 2021532

Under the Guidance of

Mr Gulshan Dhasmana

Assistant professor

Project Team ID: MP2025CSE76

Project Progress Report No: 1



Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Dehradun, Uttarakhand

2025-26



CANDIDATE'S DECLARATION

I/We hereby certify that the work which is being presented in the project progress report entitled **Blockchain-Based Freelancing Platform Using MERN** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering in the Department of Computer Science and Engineering of the Graphic Era (Deemed to be University), Dehradun shall be carried out by the undersigned under the supervision of **Mr Gulshan Dhasmana, Assistant professor**, Department of Computer Science and Engineering, Graphic Era (Deemed to be University), Dehradun.

Name1	University Roll no1	signature
Name2	University Roll no2	signature
Name3	University Rollno3	signature

The above mentioned students shall be working under the supervision of the undersigned on the **Blockchain-Based Freelancing Platform Using MERN**

Supervisor

Head of the Department

Examination

Name of the Examiners:

Signature with Date

1.

2.

Table of Contents

Chapter No.	Description	Page No.
Chapter 1	Introduction and Problem Statement	1
Chapter 2	Objectives	3
Chapter 3	Project Work Carried Out	4
Chapter 4	Future Work Plan	8
Chapter 5	Weekly Task	9
Chapter 6	– System Workflow & Flowcharts	11
Chapter 7	– Security Analysis and Formal Properties	15
•	References	15

Chapter 1

Introduction and Problem Statement

1.1 Introduction

The global workforce has undergone a massive transformation over the last decade, primarily due to digitization, globalization, and changing work preferences. Traditional employment structures, which involved fixed office hours and long-term contracts, are increasingly being replaced by freelancing, remote work, and project-based engagements. According to a 2022 report by **Statista [1]**, more than 1.5 billion people worldwide are engaged in freelancing activities.

Currently, freelancing platforms such as **Upwork [7]** and **Fiverr [8]** dominate the market. These platforms rely on centralized servers and databases to manage user data, job postings, and payments. While effective, they often act as intermediaries that control every aspect of the interaction between clients and freelancers. This project focuses on developing a robust, scalable, and secure freelancing platform using the **MERN Stack (MongoDB, Express.js, React.js, Node.js) [6]**. The MERN stack was chosen for its JSON-heavy workflow, allowing seamless data transfer between the client and server.

The current implementation of the project establishes the core web infrastructure: a secure environment where clients can post jobs, freelancers can bid via proposals, and work can be submitted and reviewed. The platform implements Role Based Access Control (RBAC) to distinguish between Clients and Freelancers, ensuring a tailored user experience.

While the current phase focuses on the Web 2.0 aspects of the platform (centralized database and API logic), the architecture is designed with future scalability in mind. The ultimate vision of this project includes the integration of **Blockchain technology [3, 10]** to introduce decentralized escrow payments, thereby addressing the issues of trust and payment security in the future phases of development..

1.2 Problem Statement

Despite the wide adoption of freelancing platforms, the ecosystem faces several critical shortcomings that this project aims to address through its initial MERN implementation and future Blockchain integration:

1. **Lack of Transparency in Workflow:** In many current systems, the status of a job (from assignment to submission) is often opaque. Our system implements a strict state-machine lifecycle (Open -> Assigned -> Submitted -> Completed) to ensure both parties are always aware of the project progress.
2. **Inefficient Proposal Management:** Freelancers often struggle to track their bids, and clients find it difficult to filter through proposals. A dedicated proposal management module has been developed to streamline this process.
3. **Data Security and Authentication:** Centralized platforms are prone to unauthorized access. We address this using bcrypt for password hashing and JWT (JSON Web Tokens) for secure, stateless authentication, ensuring that only authorized users can access sensitive routes like job creation or work submission.
4. **Future Payment Trust Issues:** A major problem in the gig economy is the lack of trust regarding payments. Clients hesitate to pay upfront, and freelancers fear nonpayment after delivery [2]. While the current system manages the logical flow of approval, the future work plan involves integrating **Ethereum-based Smart Contracts** [4] to automate this via an Escrow system, removing the need for a trusted third party.

Chapter 2 Objectives

The proposed work objectives for this Major Project are as follows:

- **To design and implement a scalable full-stack architecture** using the MERN stack (MongoDB, Express, React, Node.js) that separates concerns between the client-side interface and server-side logic.
- **To implement robust Authentication and Authorization** using JSON Web Tokens (JWT) and bcrypt encryption to secure user data and enforce Role-Based Access Control (RBAC) for Clients and Freelancers.
- **To develop a comprehensive Job Lifecycle Management system**, enabling Clients to create, update, and delete jobs, and Freelancers to view and filter open opportunities based on status and category.
- **To create an interactive Proposal and Bidding mechanism**, allowing freelancers to submit bids with cover letters and enabling clients to review and select the best candidate, automatically triggering job assignment status updates.
- **To implement a Work Submission and Review workflow**, where assigned freelancers can upload deliverables and clients can approve them to mark the job as completed.
- **To design a User Review System**, allowing both parties to rate and review each other upon project completion, calculating average ratings to build ecosystem trust.
- **To prepare the system architecture for future Blockchain integration**, specifically for a Smart Contract-based Escrow service that will handle financial transactions securely in the next phase of development.

Chapter 3

Project Work Carried Out

3.1 System Architecture

The project follows a standard Model-View-Controller (MVC) architecture adapted for the **MERN stack [6]**.

The frontend (View) is built with React.js, TypeScript, and Tailwind CSS, providing a responsive user interface.

The backend (Controller) is built on Node.js and Express.js, handling API requests and business logic. The database (Model) is MongoDB, which stores user, job, and transaction data in a document-oriented format.

- Client Side: React Context API manages the global state (User Auth, Theme). Axios is configured with interceptors to handle JWT tokens automatically.
- Server Side: Middleware is used for logging, CORS handling, and Authentication verification (AUTH Middleware).

3.2 Methodology & Algorithms

The core logic of the platform relies on state management algorithms. Below are the key algorithms implemented in this phase:

Algorithm 1: Job Lifecycle State Machine

This algorithm ensures a job cannot skip logical steps (e.g., a job cannot be "Completed" before it is "Assigned").

Input: Current Job Status (S), Action (A)

Output: New Job Status (S')

1. Initialize S = "Open"
2. IF A = "Select Proposal":
 - Verify User is Job Owner
 - Set S' = "Assigned"
 - Save selectedFreelancerID
3. IF A = "Submit Work":


```

    Verify User is SelectedFreelancer
    IF S != "Assigned" AND S != "Submitted": Return Error
    Set S' = "Submitted"
    Store File URL 4. IF
A = "Approve Work":
    Verify User is Job Owner
    IF S != "Submitted": Return
Error      Set S' = "Completed" 5.
ELSE:
    Return "Invalid Transition"

```

Algorithm 2: Secure Authentication Flow

1. User enters Email and Password.
2. Server searches User in MongoDB.
3. IF User not found -> Return 404.
4. ELSE -> Compare(InputPassword , StoredHash) using bcrypt.
5. IF Match ->
 - Generate JWT signed with Secret Key.
 - Include Payload { userId, role }.
 - Return Token to Client.
6. Client stores Token in LocalStorage.

3.3 Database Design (Schema)

The application uses MongoDB as its primary data store. The data is modeled using Mongoose with strict schema definitions to ensure data integrity. The five primary collections are defined below:

```

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    passwordHash: { type: String, required: true },
    role: { type: String, enum: ["client", "freelancer"], required: true },
    rating: { type: Number, default: 0 },
    completedProjects: { type: Number, default: 0 },
  },
  { timestamps: true }
);

```

3.3.1 User Schema

```
const jobSchema = new mongoose.Schema(  
  {  
    clientId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
    title: { type: String, required: true },  
    description: { type: String, required: true },  
    budget: { type: Number, required: true },  
    category: { type: String, required: true },  
    status: {  
      type: String,  
      enum: ["open", "assigned", "submitted", "completed"],  
      default: "open",  
    },  
    selectedFreelancer: { type: mongoose.Schema.Types.ObjectId, ref: "User" },  
  },  
  { timestamps: true }  
);
```

3.3.2 Job Schema

```
const proposalSchema = new mongoose.Schema(  
  {  
    jobId: { type: mongoose.Schema.Types.ObjectId, ref: "Job", required: true },  
    freelancerId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
    bidAmount: { type: Number, required: true },  
    coverLetter: { type: String, required: true },  
    status: { type: String, enum: ["pending", "accepted"], default: "pending" },  
  },  
  { timestamps: true }  
);
```

3.3.3 Proposal Schema

```
const reviewSchema = new mongoose.Schema(  
  {  
    jobId: { type: mongoose.Schema.Types.ObjectId, ref: "Job", required: true },  
    reviewerId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
    reviewedUser: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
    rating: { type: Number, min: 1, max: 5, required: true },  
    comment: { type: String },  
  },  
  { timestamps: true }  
);
```

3.3.4 Review Schema

```
const reviewSchema = new mongoose.Schema(
  {
    jobId: { type: mongoose.Schema.Types.ObjectId, ref: "Job", required: true },
    reviewerId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    reviewedUser: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    rating: { type: Number, min: 1, max: 5, required: true },
    comment: { type: String },
  },
  { timestamps: true }
);
```

3.3.5 Work Submission Schema

3.4 API Implementation

We have implemented RESTful API endpoints to handle data operations.

- **Authentication:**
 - POST /api/auth/register: Register new user with role. ◦ POST
 - /api/auth/login: Authenticate and receive JWT.
- **Jobs:** ◦ POST /api/jobs/create: Create a new job (Client only). ◦ GET /api/jobs/all: List all jobs with filters.
 - PUT /api/jobs/update/:id: Update job details.
- **Proposals & Work:**
 - POST /api/proposals/submit: Freelancer bids on a job.
 - PUT /api/proposals/select/:proposalId: Client selects a bid. ◦ POST
 - /api/work/submit: Freelancer uploads work. ◦ PUT
 - /api/work/approve/:id: Client approves work (Job Completion).

Chapter 4 Future Work Plan

The current system has successfully implemented the web infrastructure, database management, and business logic. The next phase of development focuses entirely on decentralization and security enhancement via Blockchain [9].

The future work plan of our project are as follows:

1	UI/UX Improvements — milestone stepper, drag-and-drop upload, wallet panel, notifications, star rating, mobile layout, dispute wizard	15 days
---	--	---------

Chapter 5

Weekly Task

The report of project work allocated by the supervisor and completed during the current semester is as follows:

Week No.	Date: From-To	Work Allocated	Work Completed (Yes/No)	Remarks	Guide Signature
1	Sept 17 - 23	Topic Selection and Synopsis Preparation			
2	Sept 24 - 30	Literature Survey and Requirement Analysis			
3	Oct 1 - 7	Literature Survey and Requirement Analysis			
4	Oct 8 - 14	Backend Setup: Node.js, Express, and Auth API			
5	Oct 15 - 21	Job Management API (CRUD) & RBAC Implementation			
6	Oct 22 - 28	Proposal & Bidding Logic Implementation			
7	Oct 29 - Nov 4	Frontend Setup: React, Tailwind, and Routing			
8	Nov 5 - 11	Integrating Frontend with Job & Auth APIs			
9	Nov 12 - 18	Work Submission & Review Module Implementation			
10	Nov 19 - 25	Testing, Debugging, and Progress Report Writing			
11	Nov 26– Dec 2	Blockchain Layer: Smart Contract Development (Escrow Logic)			

12	Dec 3–9	Sepolia Testnet Deployment & Gas Optimisation			
13	Dec 10–16	UI/UX Improvements Planning & Prototype Design			
14	Dec 17–23	System Architecture Documentation & Report Finalisation			

Chapter 6

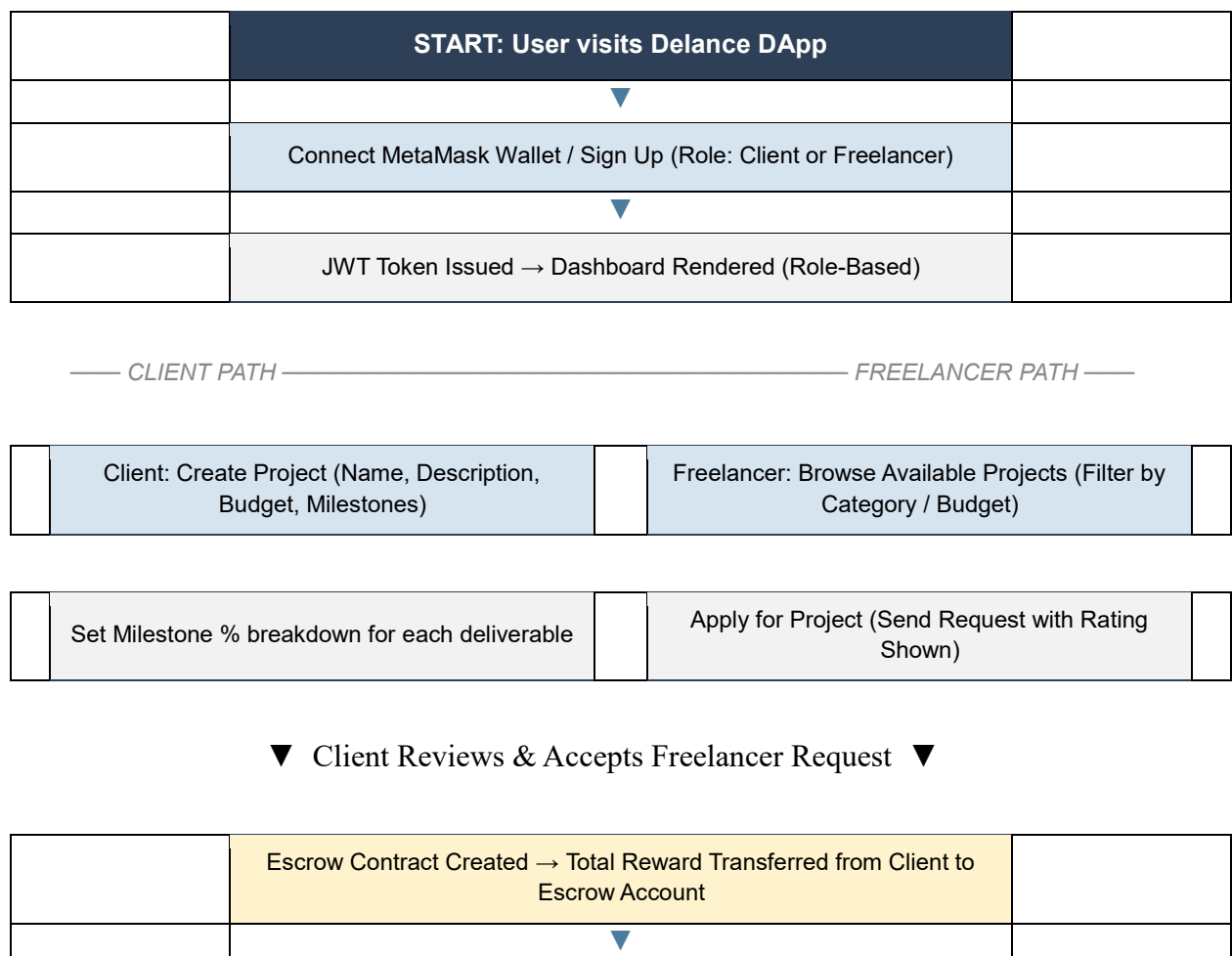
System Workflow and Flowcharts

6.1 Overview of Platform Workflow

The Delance platform orchestrates a multi-party workflow between clients and freelancers, governed by a deterministic state machine that prevents illegal state transitions. The workflow described in this chapter covers four primary flows: User Registration and Authentication, Project Posting and Application, Milestone Submission and Payment, and Dispute Resolution via Kleros arbitration. These flows map directly onto the smart contract lifecycle and RESTful API endpoints described in Chapter 3.

6.2 End-to-End Platform Workflow

Figure 6.1 presents the complete end-to-end flow from user onboarding through final payment settlement. Each step corresponds to a state transition in the Job Lifecycle State Machine (Algorithm 1, Chapter 3). The flow is divided into three phases: Onboarding, Engagement, and Settlement.



	Job Status: OPEN → ASSIGNED	
	▼	
	Freelancer Uploads Proof of Work per Milestone (File stored on IPFS → CID Generated)	
	▼	
	Client Reviews Submitted Proof	

	ACCEPTED → Milestone Payment Released from Escrow Freelancer Rating INCREASES		REJECTED → Freelancer: Accept or Dispute? (See Dispute Flow, Figure 6.2)	
--	--	--	---	--

	All Milestones Completed → Job Status: COMPLETED	
	▼	
	Mutual Rating & Review Submitted	
	▼	
	END: Transaction Record Immutably Stored on Ethereum	

Figure 6.1: End-to-End Workflow of the Delance Platform

6.3 Milestone Submission State Machine

The job lifecycle is governed by a strict four-state finite state machine: Open, Assigned, Submitted, and Completed. This design ensures that no state can be skipped, and invalid transitions return an error — a critical constraint for the integrity of the escrow logic [12]. Figure 6.2 shows the allowed transitions between states.

OPEN	→	ASSIGNED	→	SUBMITTED	→	COMPLETED
Project created by Client		Freelancer selected; Escrow funded		Freelancer uploads milestone proof (IPFS CID)		Client approves; Escrow released; Ratings updated

Figure 6.2: Job Lifecycle State Machine — Allowed Transitions

6.4 Dispute Resolution Flowchart

When a client rejects a freelancer's milestone submission, the platform provides two resolution paths: the freelancer may accept the rejection and revise the work (resulting in a rating penalty and resubmission), or the freelancer may invoke the Kleros decentralised arbitration service

[9]. Kleros appoints an anonymous juror pool that reviews the IPFS-stored proof files and issues a binding ruling on fund distribution. Figure 6.3 details this branching workflow.

Client REJECTS Submission				
▼				
Freelancer Decision				
▼ Accept Rejection				▼ Raise Dispute
Revise & Resubmit Work				Kleros Arbitration Invoked
▼ Rating Decreases				▼
Back to Milestone Submission				Anonymous Arbitrator Reviews Files (IPFS CID)
				▼
				Decision: Client Valid → Funds Stay Freelancer Valid → Funds Released
				▼
				Ratings Updated for Both Parties

Figure 6.3: Dispute Resolution Flowchart via Kleros Arbitration

6.5 Authentication Workflow

User authentication follows a stateless JWT-based model, as described in Algorithm 2 (Chapter 3). Upon successful login, the server issues a signed JWT containing the user ID and role. This token is stored in the client's browser and is automatically injected into all subsequent API requests by the Axios interceptor. On the server, the AUTH Middleware validates the token signature and decodes the payload before permitting access to protected routes.

User: Enter Email + Password	→	Server: Lookup User in MongoDB	→	bcrypt.compare(input, hash)
				▼
Client stores JWT in memory	←	Server generates signed JWT { userId, role }	←	MATCH → Issue JWT NO MATCH → 401 Unauthorized
▼				
Axios attaches Bearer Token to all subsequent API requests				

Figure 6.4: Authentication Workflow — JWT Issuance and Usage

6.6 Rating Calculation Algorithm

Ratings are calculated as a weighted average across all completed transactions, accounting for three possible outcomes: full acceptance (positive weight), acceptance with delay (neutral-to-negative weight), or arbitration ruling. This automated calculation prevents manipulation by either party, as ratings are updated programmatically by the smart contract upon escrow settlement rather than entered manually. Let R be the updated rating, where R_{old} is the previous rating, n is the number of prior transactions, and w is the weight of the new transaction outcome ($w = 1.0$ for acceptance, $w = 0.7$ for delayed acceptance, $w = 0.0$ for rejected work).

$$R_{new} = (R_{old} \times n + w) \div (n + 1)$$

Formula 6.1: Weighted Average Rating Update

6.7 Summary

This chapter presented the complete operational workflows of the Delance platform through four detailed diagrams covering the end-to-end transaction lifecycle (Figure 6.1), the job state machine (Figure 6.2), the Kleros dispute resolution path (Figure 6.3), and the JWT authentication flow (Figure 6.4). These flows collectively demonstrate how the MERN backend and Blockchain layer cooperate to deliver a trustless, automated, and transparent freelancing experience — addressing the core problem statement identified in Chapter 1. The automated rating formula (Formula 6.1) further ensures that reputation signals are tamper-resistant and objective [2].

Chapter 7

Security Analysis and Formal Properties

7.1 Overview

This chapter presents a formal security analysis of the Delance smart contract system. The smart contract is modelled as a deterministic state-transition machine and evaluated against standard adversarial capabilities on the Ethereum network. Security proofs are provided for reentrancy resistance, unauthorized fund release prevention, double-spend prevention, and escrow balance conservation. Additionally, a game-theoretic analysis is presented to demonstrate that honest participation constitutes the rational equilibrium strategy for both clients and freelancers [2][3].

7.2 System as a Deterministic State Machine

The Delance smart contract is formally modelled as a deterministic transition system $M = (S, A, T)$, where S is the set of all possible contract states, A is the set of valid actions that may be invoked, and $T : S \times A \rightarrow S$ is the deterministic transition function. For any transaction Tx in A , the next state is given by: $St+1 = T(St, Tx)$. Because Ethereum execution is fully deterministic, identical inputs always produce identical state transitions — ensuring predictable, verifiable contract behaviour. Every project passes through the following defined states: Created $\rightarrow$ Funded $\rightarrow$ InProgress $\rightarrow$ Submitted $\rightarrow$ Approved $\rightarrow$ Released $\rightarrow$ Closed. Any undefined transition causes the transaction to revert automatically.

7.3 Adversary Model

We assume a probabilistic polynomial-time adversary with the following capabilities: the ability to submit arbitrary transactions to the network, to deploy malicious smart contracts, and to attempt reentrant calls during fund release. The adversary can observe all public blockchain data. However, the adversary cannot break Ethereum's cryptographic primitives and cannot reverse finalized blocks with non-negligible probability. Ethereum consensus is assumed to operate under a majority-honest assumption, meaning the adversary controls less than 50% of network stake.

7.4 Security Properties and Proofs

7.4.1 Reentrancy Resistance

Reentrancy occurs when an external call allows recursive re-entry into a contract function before state updates are finalized. In the Delance escrow release function, state transitions are executed before any external Ether transfer. Specifically, the milestone state is updated to Released before the transfer instruction is executed. If an adversarial freelancer contract attempts re-entry during the transfer, the condition check (milestone state must equal

Submitted) fails because the state has already been updated. Therefore, recursive release attempts are always reverted, and the contract is fully resistant to reentrancy attacks [2].

7.4.2 Unauthorized Release Prevention

Fund release is conditioned on three simultaneous requirements: (1) the transaction sender must be the project's client address, (2) the milestone state must equal Submitted, and (3) the milestone must not have been previously released. The transfer target — the freelancer address — is stored immutably at project creation and never modified thereafter. The release function accepts no external address parameter. Therefore, no address other than the assigned freelancer can receive escrowed funds under any valid execution path [2].

7.4.3 Double-Spend Prevention

Each milestone can be released at most once. Release requires the milestone state to equal Submitted. Upon successful execution, the state is immediately updated to Released. All subsequent calls to release the same milestone fail the state condition check. Therefore, the release count for any milestone M_i satisfies $\text{ReleaseCount}(M_i) \leq 1$, and double-spending of milestone funds is provably impossible.

7.4.4 Escrow Balance Invariant

At all reachable contract states, the contract's Ether balance must equal the sum of all locked milestone funds across active projects. Initially the contract balance is zero. Upon project funding, both the balance and locked funds increase by the same budget amount. Upon milestone release, both decrease by the same milestone amount simultaneously. By induction, the invariant $\text{ContractBalance} = \text{Sum}(\text{LockedFunds})$ holds at all times, ensuring that no Ether can be created or destroyed within the contract.

Property	Name	Formal Statement
P1	No Early Release	Funds cannot be released unless milestone state = Approved
P2	Single Release	$\text{ReleaseCount}(M_i) \leq 1$ for all milestones M_i
P3	Balance Conservation	$\text{ContractBalance} = \text{Sum of all LockedFunds across active projects at all times}$
P4	Eventual Completion	If both parties behave honestly: Submitted state eventually leads to Released state
P5	Progress Guarantee	If a milestone is submitted and approved, fund release executes within the same transaction

Table 7.1: Formal Safety and Liveness Properties of the Escrow Contract

7.5 Front-Running and DoS Analysis

Since milestone release requires the client's authorisation (msg.sender must equal the client address), no third party can front-run the release transaction. Even if a pending transaction is observed in the mempool, only the client can execute the approval. Fund redirection via front-running is therefore impossible. Regarding denial-of-service, the release function contains no unbounded loops and executes all operations in constant time $O(1)$. Gas exhaustion attacks against the release function are infeasible because gas consumption is predictable and bounded at a constant value regardless of contract state [2].

7.6 Game-Theoretic Incentive Analysis

The client-freelancer interaction is modelled as a sequential game. The freelancer first chooses an effort level (honest or dishonest work), after which the client observes the submission and chooses to approve or reject. Using backward induction, a rational client will approve if and only if the work is honest, since approval of honest work yields positive utility while rejection of honest work wastes the client's payment without recovering funds. Anticipating this, a rational freelancer earns the milestone payment V minus effort cost C_f by working honestly, but earns nothing (only incurring gas cost G) by working dishonestly. Since $V - C_f - G > -G$ whenever $V > C_f$, honest work strictly dominates. Therefore, (HonestWork, Approve) constitutes a Subgame Perfect Nash Equilibrium under standard rationality assumptions.

Strategy Combination	Freelancer Utility	Client Utility	Outcome
Honest Work + Approve	$V - C_f - G$	$B - V - G_c$	Nash Equilibrium $\square$
Honest Work + Reject	$-G$	$-G_c$	Irrational for client
Dishonest Work + Reject	$-G$	$-G_c$	No incentive to cheat
Dishonest Work + Approve	$V - G$	$B - V - G_c$ (low B)	Irrational for client

Table 7.2: Game-Theoretic Payoff Matrix for Client-Freelancer Interaction

7.7 Probabilistic Finality and Consensus Security

Ethereum provides probabilistic finality. The probability that an adversary can reverse a transaction after k confirmations is bounded by $\text{Preorg}(k) \approx e^{(-\gamma k)}$, where γ depends on the difference between honest and adversarial block production rates. As k increases, this probability approaches zero. With an average Ethereum block time of approximately 12 seconds, $k = 12$ confirmations provides finality in approximately 144 seconds with negligibly small reversal probability. Under the majority-honest assumption (adversary controls less than 50% of network stake), finalized escrow releases cannot be reversed with non-negligible probability [3]. Attacking the escrow would require controlling majority stake, the cost of which

far exceeds the value of any typical freelancing contract — making attacks economically irrational.

7.8 Computational Complexity of Contract Operations

The time complexity of each core contract operation has been formally analysed. Project creation runs in $O(k)$ time where k is the number of milestones, as the contract iterates once over the milestone list to initialise each entry. Escrow funding, milestone submission, and milestone release all execute in $O(1)$ constant time, as each involves only fixed-cost mapping lookups, single state updates, and one Ether transfer. There are no unbounded loops in any payment-critical function. Total storage complexity grows as $O(n \times k)$ where n is the number of projects, which must be managed through Layer 2 migration or storage pruning for large-scale deployment.

Operation	Time Complexity	Notes
Project Creation	$O(k)$	k = number of milestones; iterates once over milestone list
Escrow Funding	$O(1)$	Mapping lookup + balance update; no iteration
Milestone Submission	$O(1)$	Single index access + IPFS hash store + state update
Approve & Release	$O(1)$	Condition check + state update + single Ether transfer
Total Storage	$O(n \cdot k)$	n = total projects, k = avg milestones per project

Table 7.3: Computational Complexity of Core Smart Contract Operations

7.9 Limitations

Despite the strong security guarantees provided by the smart contract design, several practical limitations exist. First, gas cost volatility on Ethereum Layer 1 means execution costs are unpredictable, creating economic uncertainty compared to fixed-percentage fees charged by centralised platforms. Second, all project metadata stored on-chain is publicly visible — participant addresses, funding amounts, and milestone releases can be observed by any network participant. Third, the current design does not include a native decentralised arbitration layer; if a client refuses approval despite valid freelancer work, funds remain locked until a timeout or mutual agreement is reached. Fourth, permanent on-chain storage grows as $O(n \times k)$, limiting long-term scalability without Layer 2 migration.

References

- [1] Statista Research Department, "Freelance workforce in the United States – Statistics & Facts," *Statista*, 2022.
- [2] A. R. Bharambe, S. K. Sharma, and M. Ghosh, "Blockchain-based freelancing platforms: Towards decentralized trust and transparency," *Proc. IEEE Int. Conf. on Blockchain and Distributed Systems*, pp. 45–52, 2021.
- [3] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
- [4] V. Buterin, "Ethereum White Paper: A Next-Generation Smart Contract and Decentralized Application Platform," 2014.
- [5] K. Chodorow, *MongoDB: The Definitive Guide*, 3rd ed., O'Reilly Media, 2019.
- [6] B. Shroff, *Full-Stack React Projects: Modern Web Development using React, Node, Express, and MongoDB*, Packt Publishing, 2018.
- [7] Upwork Global Inc., "About Upwork," Available: <https://www.upwork.com>. Accessed: 25 Sept. 2025.
- [8] Fiverr International Ltd., "How Fiverr Works," Available: <https://www.fiverr.com>. Accessed: 25 Sept. 2025.
- [9] J. Al-Breiki, M. H. ur Rehman, K. Salah, and D. Svetinovic, "Trustworthy blockchain oracles: Review, comparison, and open research challenges," *IEEE Access*, vol. 8, pp. 85675–85685, 2020.
- [10] M. Crosby, P. Pattanayak, S. Verma, and V. Kalyanaraman, "Blockchain technology: Beyond bitcoin," *Applied Innovation Review*, vol. 2, pp. 6–19, June 2016.
- [11] J. Lim et al., "Blocklance - A Blockchain Powered Freelancing Platform," *International Journal of Scientific Research in Engineering and Management*, vol. 09, no. 11, Nov. 2025.
- [12] R. Modi, *Solidity Programming Essentials: A Beginner's Guide to Build Smart Contracts for Ethereum*, Packt Publishing, 2018.
- [13] I. Bashir, *Mastering Blockchain: Distributed Ledger Technology, Decentralization, and Smart Contracts*, 2nd ed., Packt Publishing, 2018.
- [14] D. Drescher, *Blockchain Basics: A Non-Technical Introduction in 25 Steps*, Apress, 2017.
- [15] V. Subramanian, *Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node*, 2nd ed., Apress, 2019.
- [16] Banks, A. and Porcello, E., *Learning React: Modern Patterns for Developing React Apps*, 2nd ed., O'Reilly Media, 2020.